

HOSTEL MAINTENANCE & COMPLAINT TRACKER

PROJECT REPORT

Project Information	Details
Course	Programming in Java
Academic Year	2026–27
Submitted By	AAKARSHA PARASHAR
Registration Number	25BAI10492

Declaration

I, Aakarsha Parashar, hereby declare that the project titled “Hostel Maintenance & Complaint Tracker” has been developed as an academic project for the Programming in Java course. The project demonstrates the practical implementation of Java programming concepts such as Object-Oriented Programming, collections, file handling, exception handling, methods, and input validation.

Acknowledgement

I would like to express my sincere gratitude to my faculty member for providing guidance and support throughout the development of this project. I am also thankful to my institution and classmates for their valuable suggestions and encouragement. This project helped me understand how Java programming concepts can be applied to solve a practical problem in hostel management.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Functional Requirements
6. Non-Functional Requirements
7. System Architecture
8. Design Diagrams
9. Design Decisions & Rationale
10. Implementation Details
11. Storage Design
12. Working of the System
13. Screenshots / Results
14. Testing Approach
15. Challenges Faced
16. Learnings & Key Takeaways
17. Future Enhancements
18. Conclusion
19. References

1. Introduction

Hostel students regularly face maintenance problems such as damaged electrical equipment, water leakage, cleaning issues, broken furniture, internet problems, and other room-related issues. Managing such complaints manually can make it difficult to maintain records and track whether a complaint has been resolved.

The Hostel Maintenance & Complaint Tracker provides a simple Java-based solution where students can submit complaints and track their status and priority. The project is implemented as a console-based application, making it simple to operate while demonstrating important Java programming concepts.

Project Overview

The application provides a menu-driven interface for complaint management. A complaint contains information such as complaint ID, student name, room number, category, description, priority, and status. Records are kept in an ArrayList during execution and are persisted in a text file so that saved complaints can be loaded again.

Java Concepts Demonstrated

- Classes and objects
- Encapsulation using private fields and getters/setters
- Constructors and methods
- ArrayList collection
- File handling using readers/writers
- Exception handling and input validation
- String processing for search operations
- Modular program structure

2. Problem Statement

In a hostel environment, maintenance complaints may be recorded informally or manually. This can create difficulty in maintaining complaint records, identifying high-priority issues, tracking complaint status, finding a particular complaint, and retaining older records.

Problems Addressed

- Difficulty maintaining organized complaint records.
- Difficulty identifying and tracking high-priority maintenance issues.
- Difficulty checking whether complaints are submitted, in progress, or resolved.
- Difficulty searching for complaints using student, room, category, or description.
- Risk of losing records when information is maintained only temporarily.
- Lack of a simple summary of complaint information.

3. Objectives

1. Develop a simple system for recording hostel maintenance complaints.
2. Store student and room details along with each complaint.
3. Categorize complaints according to maintenance type.
4. Assign a priority level to each complaint.
5. Track the status of complaints.
6. Provide search functionality for existing records.
7. Allow complaint information to be updated or deleted.
8. Provide pending, resolved, and summary views.
9. Use file handling to preserve complaint data between program executions.
10. Demonstrate Java programming and Object-Oriented Programming concepts.

4. Scope of the Project

The current project focuses on a console-based hostel maintenance complaint management system. It is designed as an academic Java project and concentrates on the core operations needed to create and manage complaint records.

Included in the Current Scope

- Submit a new complaint.
- View all stored complaints.
- Search complaints.
- Update complaint status.
- Update complaint priority.
- Delete a complaint.
- View pending complaints.
- View resolved complaints.
- View complaint summary/statistics.
- Save and load records using a text file.

Outside the Current Scope

- Graphical user interface.
- Online/cloud deployment.
- Relational database integration.
- Separate student and administrator login.
- Automatic email or mobile notifications.

5. Functional Requirements

The VITyarthi guidelines require at least three major functional modules, a clear input/output structure, and a logical user workflow. The project provides multiple complaint-management modules.

Module	Input	Processing	Output
Complaint Submission	Student name, room, category, description, priority	Validate data, create complaint, assign ID, store record	Confirmation and complaint ID
Complaint Viewing	Menu selection	Read complaints from in-memory collection	Formatted complaint records
Complaint Search	ID/student/room/category/description	Compare search text with stored records	Matching complaints
Complaint Update	Complaint ID and new value	Find record and change status/priority	Updated record/confirmation
Complaint Deletion	Complaint ID	Find and remove record	Deletion confirmation
Complaint Status Views	Menu selection	Filter records by status	Pending/resolved complaints
Complaint Summary	Menu selection	Count and summarize records	Complaint statistics
File Persistence	Stored complaint list	Read/write text file	Saved records available after restart

6. Non-Functional Requirements

Requirement	Description
Usability	The menu-driven interface should be simple to understand and operate.
Reliability	Complaint records should be saved to a text file so stored data can be loaded again.
Performance	For a normal academic-sized complaint list, searching and updating records should be responsive.
Maintainability	Functionality is divided into separate classes such as Complaint, ComplaintManager, FileManager, and InputValidator.
Error Handling	Invalid numeric and menu inputs are handled through validation and exception handling.
Resource Efficiency	The application uses an ArrayList and a simple text file without requiring a database server.
Portability	The application is intended to run on systems with a compatible Java JDK.

7. System Architecture

The architecture follows a simple modular structure. The Main class provides the user interface and menu handling. InputValidator checks user input. ComplaintManager performs complaint operations, Complaint stores the complaint data, and FileManager handles persistent text-file storage.

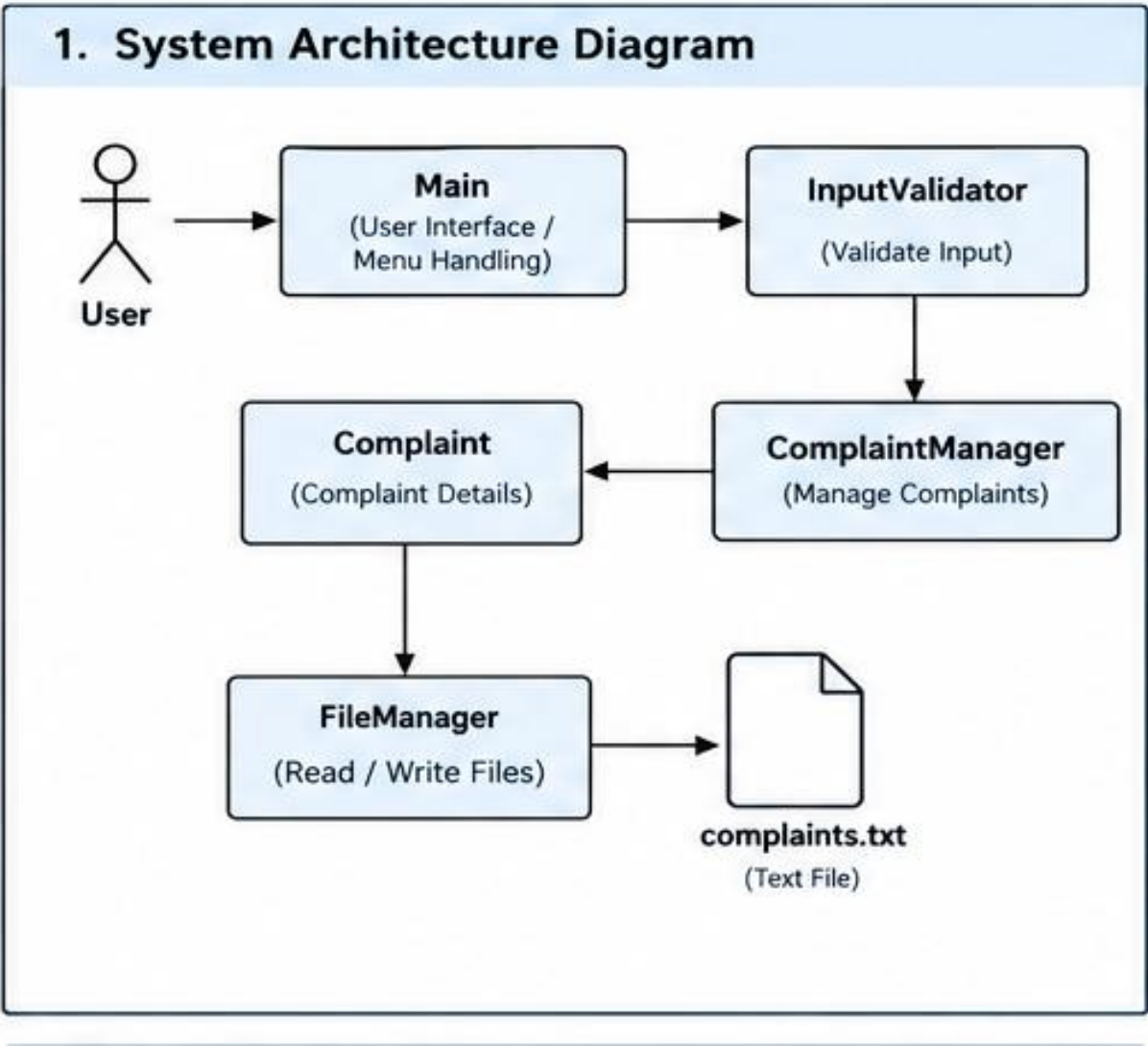


Figure 1: System Architecture Diagram

Architecture Components

Component	Responsibility
Main.java	Controls program execution, menu display, and user interaction.
InputValidator.java	Validates strings, integers, categories, priorities, and status choices.
ComplaintManager.java	Manages the complaint collection and CRUD/search/status operations.
Complaint.java	Represents a single complaint and its attributes.
FileManager.java	Reads complaint records from and writes records to the text file.
complaints.txt	Persistent text-file storage for complaint records.

8. Design Diagrams

The following diagrams are included according to the design and documentation requirements: use case, workflow, sequence, class/component, system architecture, and storage design.

8.1 Use Case Diagram

The Student/User interacts with the system through the major complaint-management functions: submitting, viewing, searching, updating, deleting complaints, and viewing the summary.

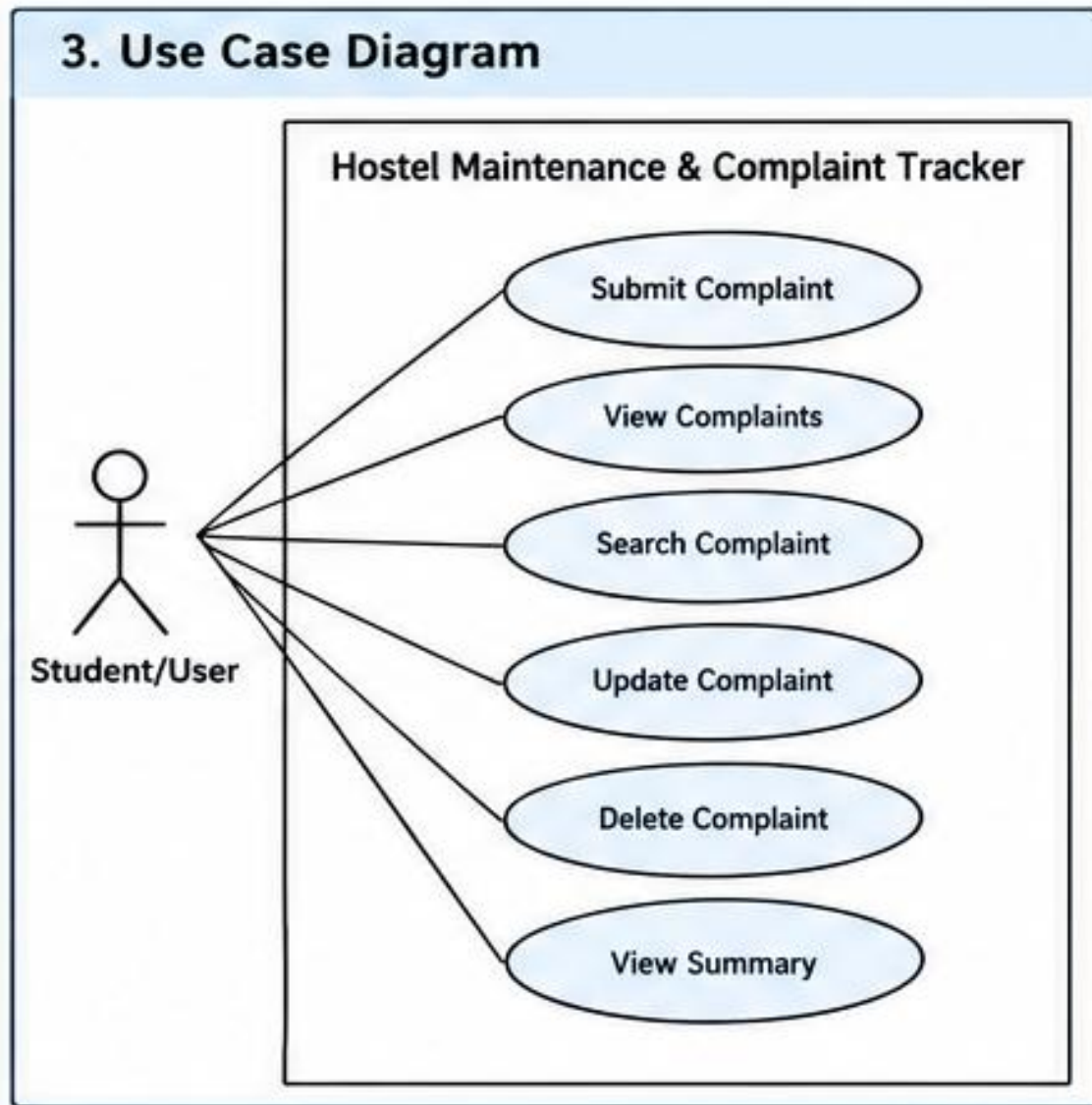


Figure 2: Use Case Diagram

8.2 Process Flow / Workflow Diagram

The workflow begins at the main menu. The selected operation is validated and processed. Depending on the operation, the system either reads/writes the file or processes the complaint collection in memory. The result is displayed and the user can continue or exit.

2. Process Flow / Workflow Diagram

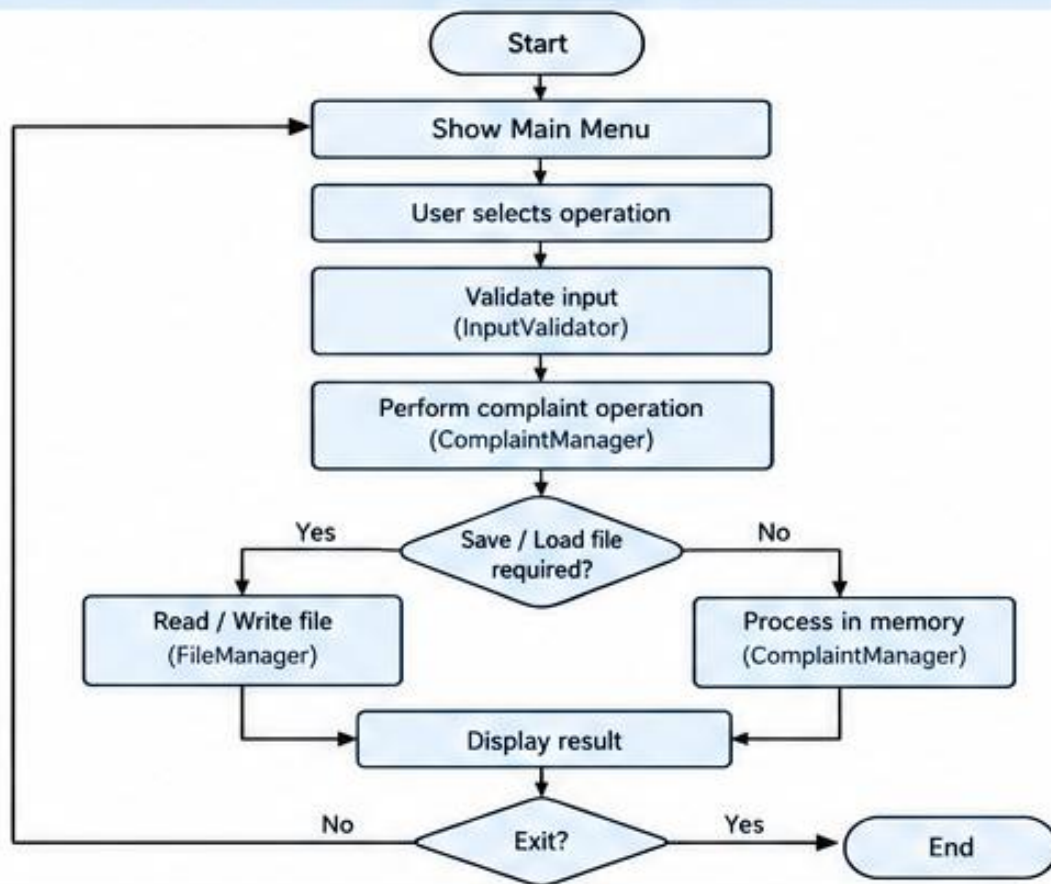


Figure 3: Process Flow / Workflow Diagram

8.3 Class Diagram

The class diagram shows the relationships among Main, InputValidator, ComplaintManager, FileManager, and Complaint. ComplaintManager manages multiple Complaint objects and uses FileManager for persistence.

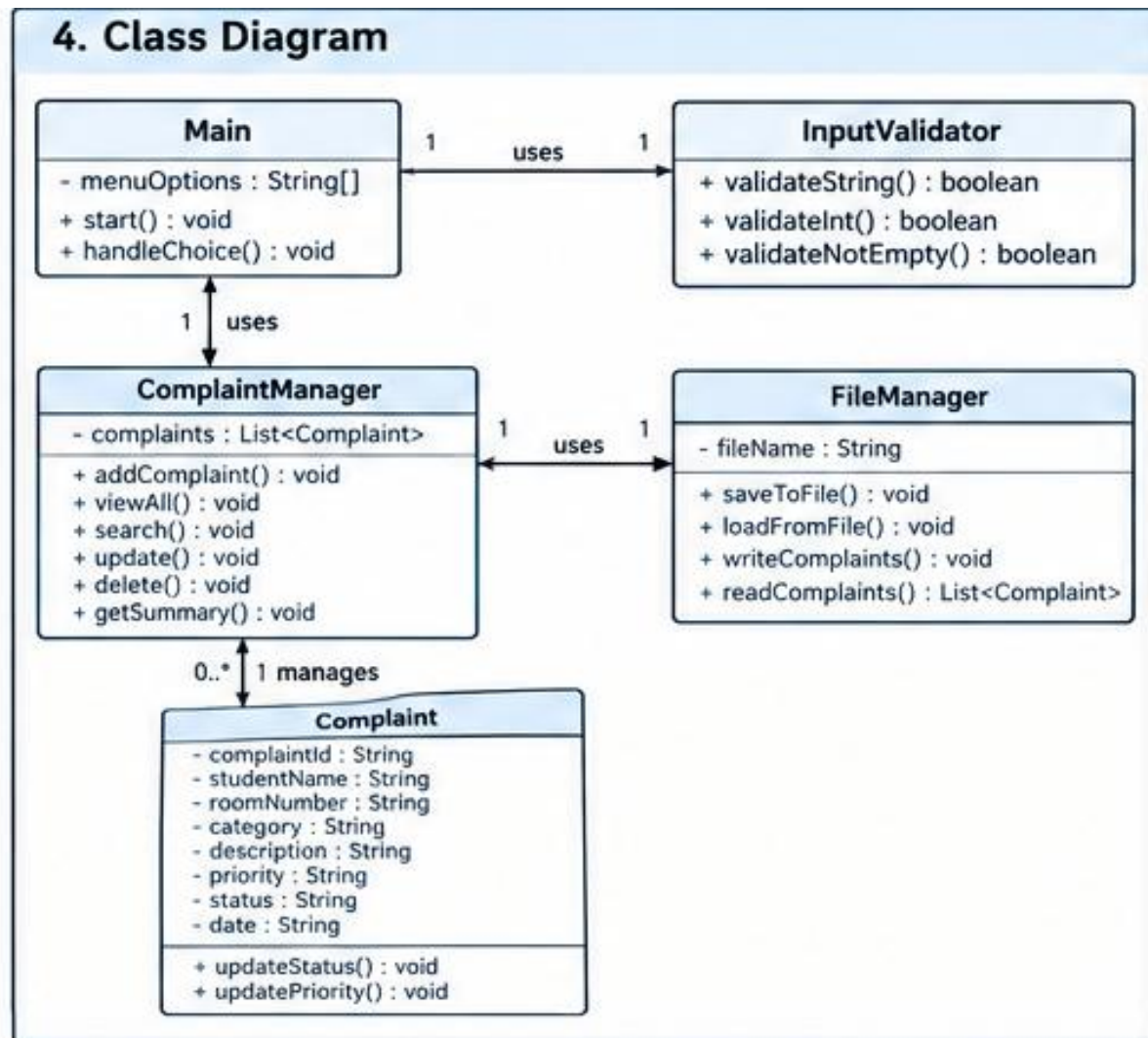


Figure 4: Class Diagram

8.4 Sequence Diagram – Submit Complaint

The sequence illustrates the submission process: the user selects Submit Complaint, Main requests validation, ComplaintManager creates the complaint object, and FileManager saves the record. A confirmation is then shown.

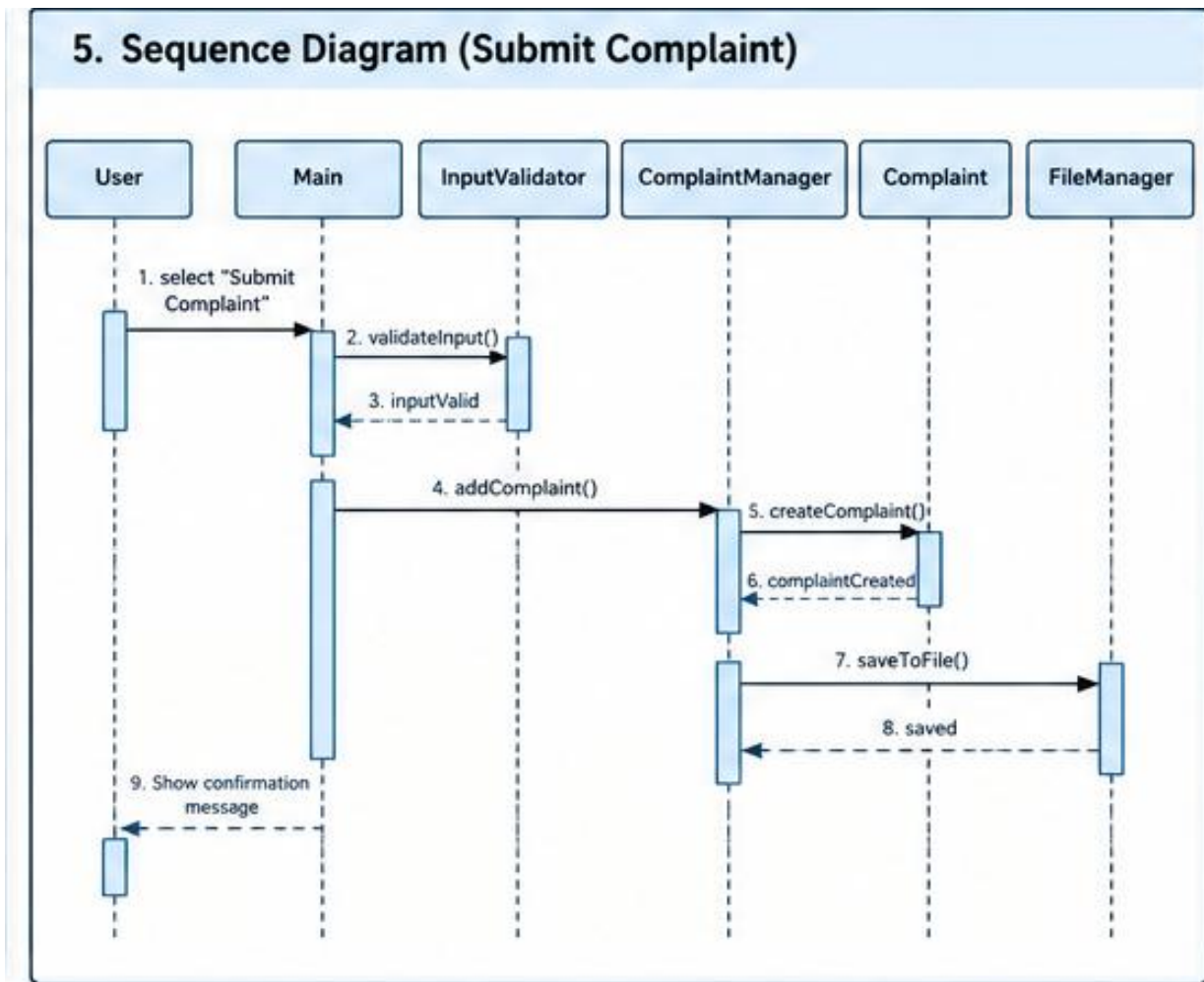


Figure 5: Sequence Diagram – Submit Complaint

8.5 Storage Design

The system uses a text file named complaints.txt. Complaint/FileManager is responsible for reading and writing the complaint records.

6. Storage Design

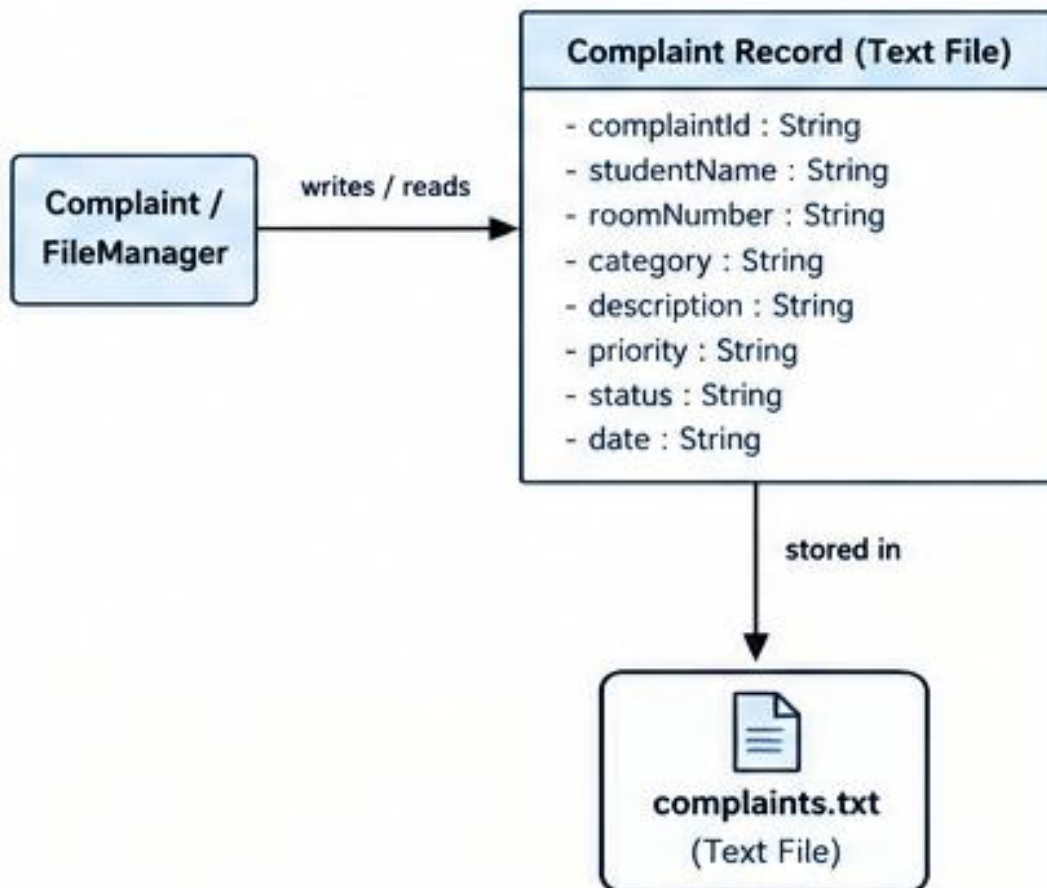


Figure 6: Storage Design

9. Design Decisions & Rationale

Decision	Rationale
Console-based interface	Keeps the academic implementation focused on Java programming and core logic without adding GUI complexity.
ArrayList<Complaint>	Provides dynamic storage for a changing number of complaint objects during program execution.
Separate Complaint class	Keeps complaint data organized and demonstrates object-oriented design.
ComplaintManager class	Centralizes complaint operations such as add, search, update, delete, filtering, and summary.
FileManager class	Separates persistence logic from complaint-management logic.
InputValidator class	Keeps validation code separate and makes menu operations easier to maintain.
Text-file persistence	Provides simple persistent storage without requiring a database server.
Status and priority fields	Allow users to track the progress and urgency of maintenance issues.

10. Implementation Details

10.1 Core Classes / Files

File	Purpose	Main Concepts
Main.java	Runs the application and handles menu operations.	Methods, control flow, input handling
Complaint.java	Defines complaint data and object behavior.	Class, object, constructor, encapsulation
ComplaintManager.java	Manages complaint records and operations.	ArrayList, methods, searching, CRUD
FileManager.java	Loads and saves complaint records.	File handling, BufferedReader/Writer
InputValidator.java	Validates user input and menu selections.	Methods, validation, exception handling

10.2 Complaint Attributes

Attribute	Type	Description
complaintId	String	Unique complaint identifier.
studentName	String	Name of the student submitting the complaint.
roomNumber	String	Hostel room associated with the complaint.
category	String	Maintenance category such as Electrical or Plumbing.
description	String	Description of the maintenance issue.
priority	String	Low, Medium, or High.
status	String	Submitted, In Progress, or Resolved.
date	String	Date field represented in the design/storage model.

10.3 Complaint Categories

- Electrical
- Plumbing
- Cleaning
- Furniture
- Internet
- Room Maintenance
- Other

10.4 Complaint Status

- Submitted – complaint has been recorded.
- In Progress – complaint is being addressed.
- Resolved – maintenance issue has been completed.

10.5 Priority Levels

- Low
- Medium
- High

11. Storage Design

The project does not use a relational database. Instead, complaint records are stored in the text file data/complaints.txt. The file is loaded when the application starts and updated when complaint data changes.

11.1 Text-File Record Format

The storage design uses a pipe-separated record structure. A representative record is:

1001|Aakriti|B-204|Electrical|Ceiling fan is not working|High|In Progress

11.2 Storage Schema

Field	Meaning
Complaint ID	Unique identifier generated for a complaint.
Student Name	Student who submitted the complaint.
Room Number	Hostel room number.
Category	Type of maintenance issue.
Description	Detailed issue description.
Priority	Low, Medium, or High.
Status	Submitted, In Progress, or Resolved.

Because the implementation uses a text file rather than a relational database, a conventional relational ER diagram is not directly applicable. The supplied Storage Design diagram and the above record schema document the actual persistence model used by this project.

12. Working of the System

12.1 General Workflow

11. Start the application.
12. Load previously saved complaints from complaints.txt.
13. Display the main menu.
14. Accept the user's operation choice.
15. Validate the required input.
16. Perform the selected complaint operation.
17. Save changes to the text file whenever persistence is required.
18. Display the result to the user.
19. Return to the menu until Exit is selected.

12.2 Complaint Submission Algorithm

20. Select Submit Complaint.
21. Enter student name and room number.
22. Select a complaint category.
23. Enter the complaint description.
24. Select the priority level.
25. Generate the next complaint ID.
26. Create a Complaint object.
27. Add the object to the complaint collection.
28. Save the updated records to the text file.
29. Display the complaint ID and confirmation message.

13. Screenshots / Results

The following screenshots are included from the provided example output document. They demonstrate the console menu, complaint submission, generated complaint IDs, viewing all complaints, and program exit.

13.1 Output Screenshot 1

EXAMPLE COMPLAINTS
(SCREENSHOT OF OUTPUT OF CODE)

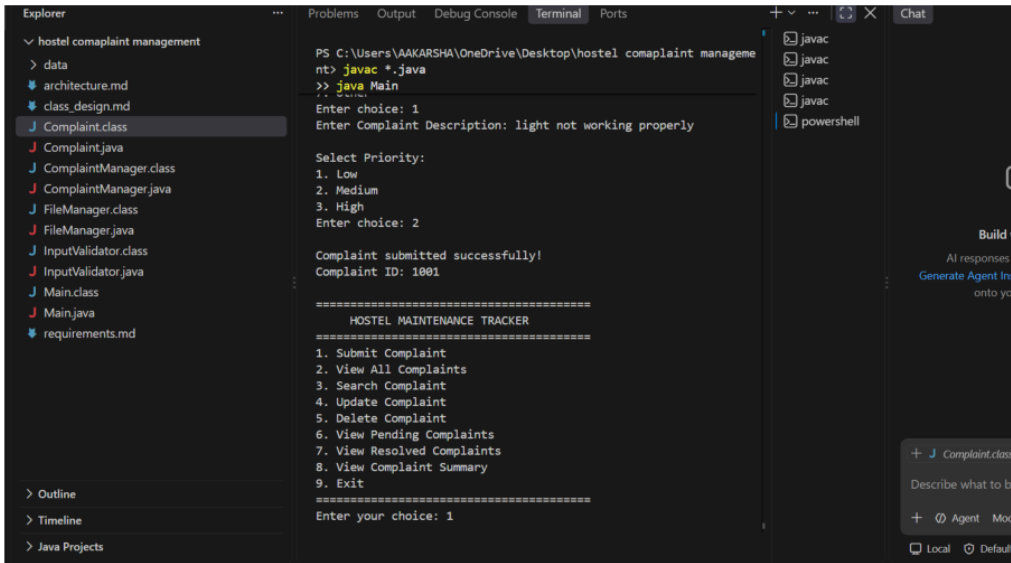
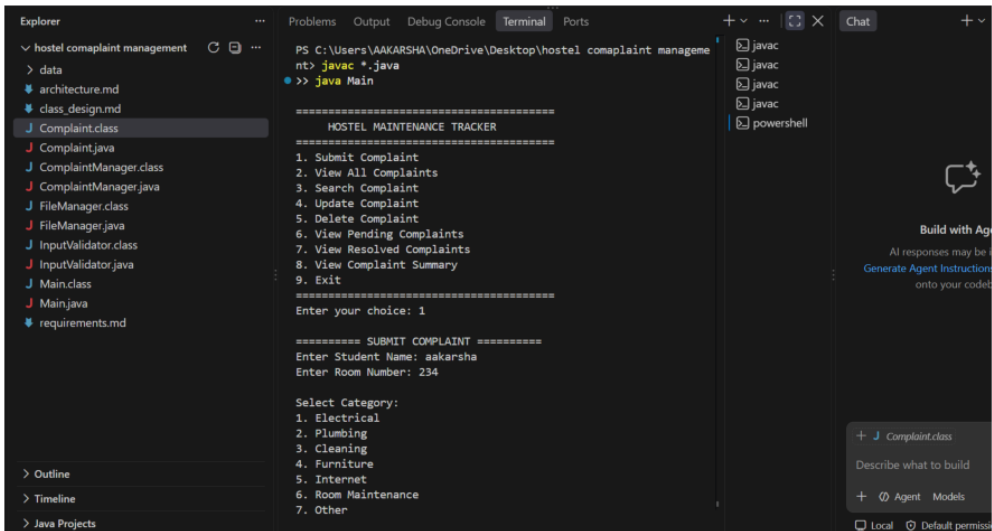


Figure 7: Example Output Screenshot 1

13.2 Output Screenshot 2

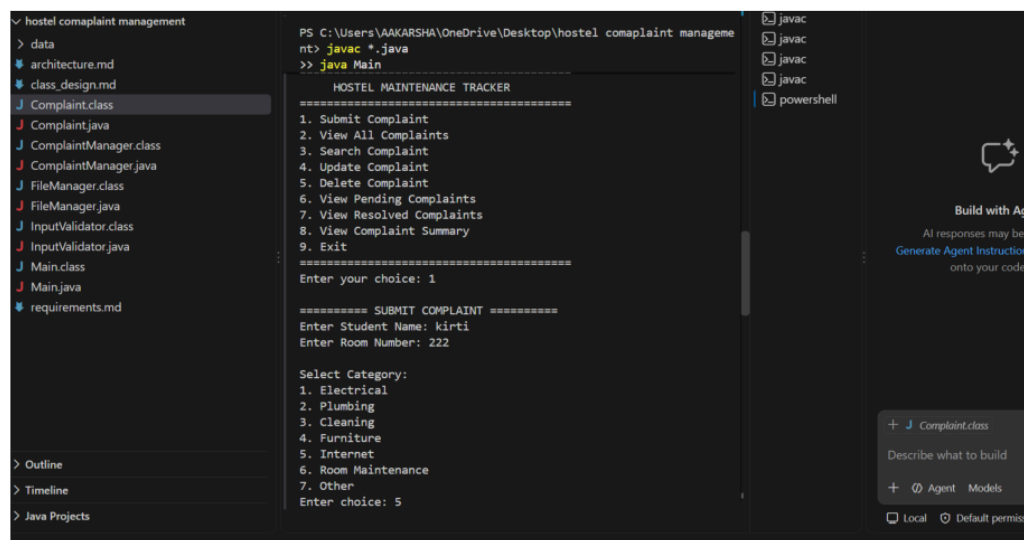
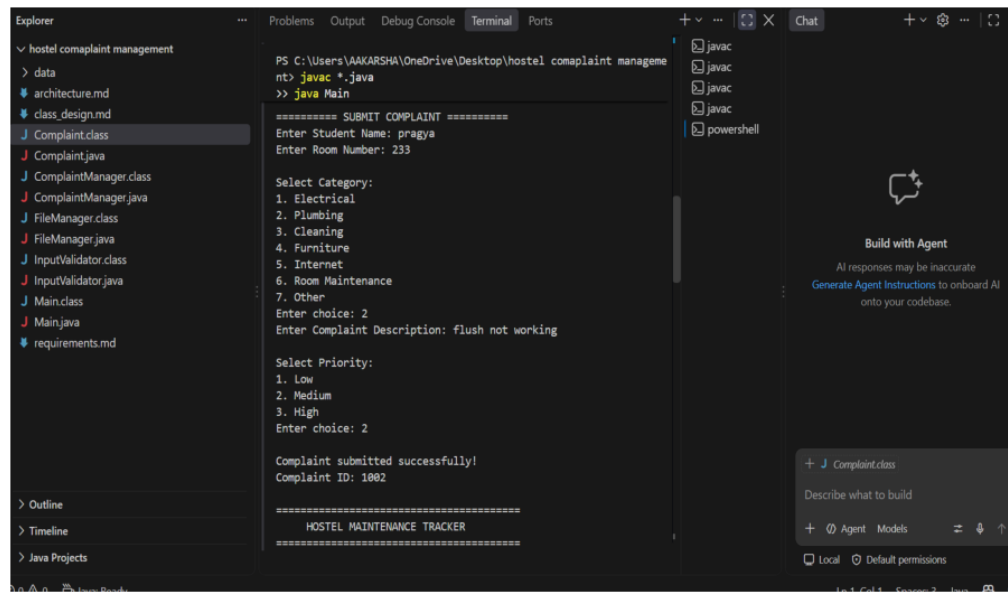


Figure 8: Example Output Screenshot 2

13.3 Output Screenshot 3

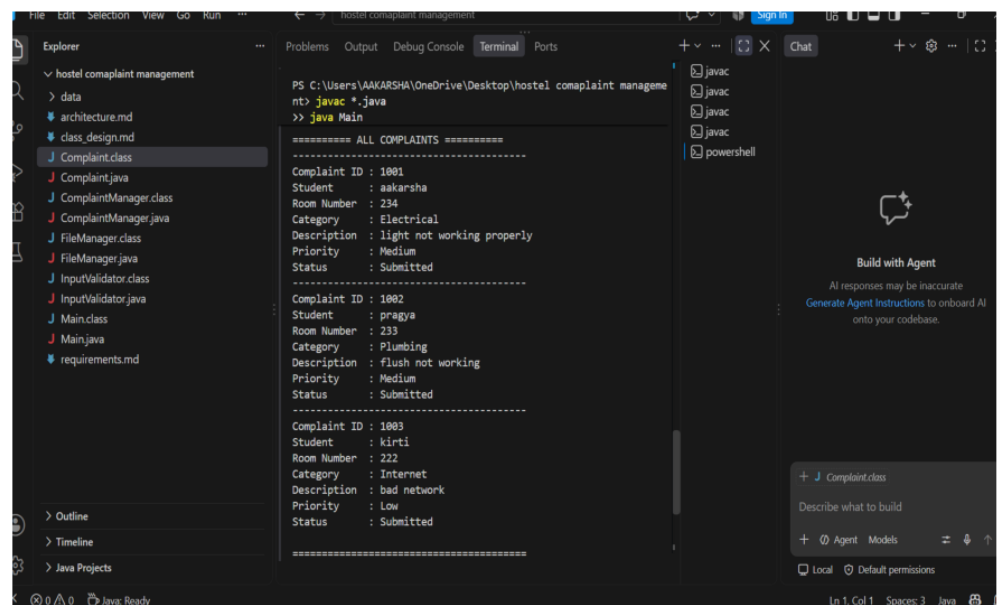
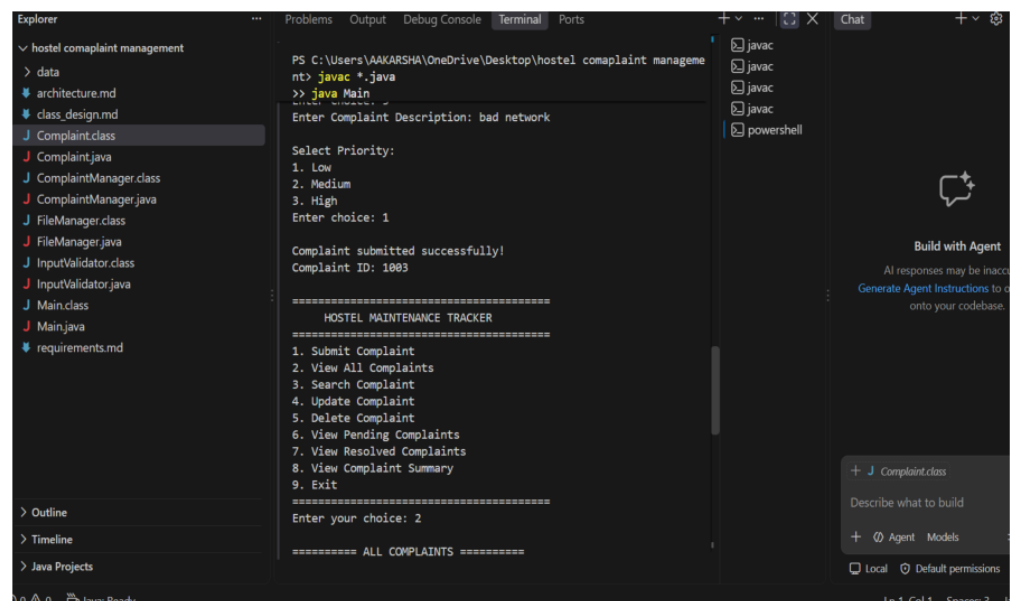


Figure 9: Example Output Screenshot 3

13.4 Output Screenshot 4

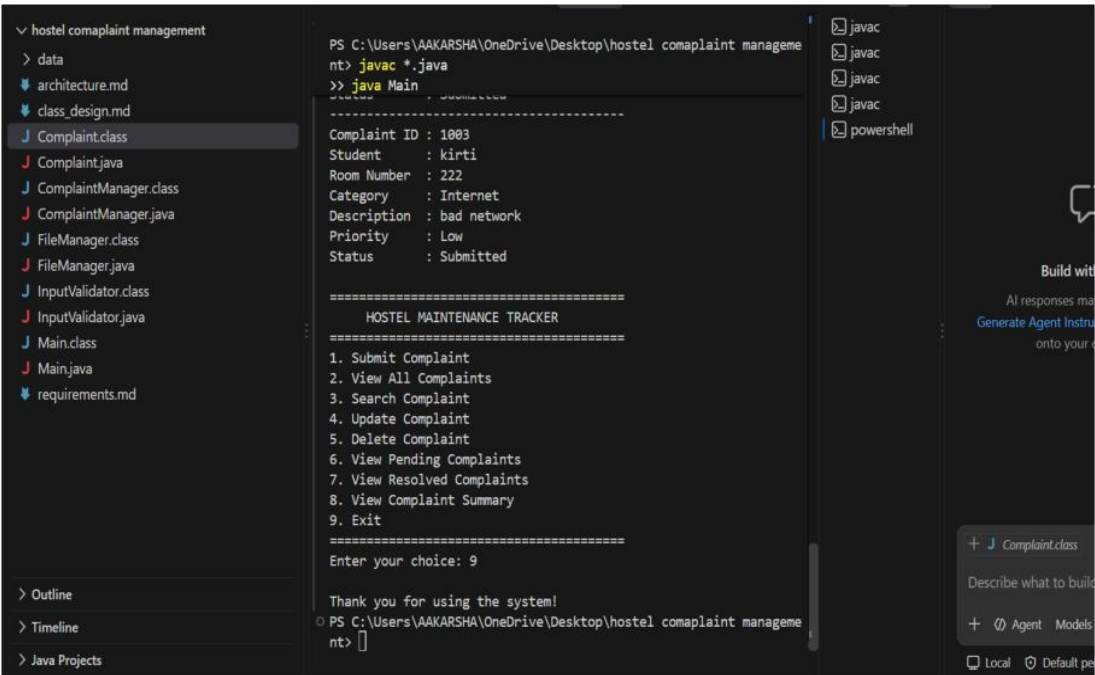


Figure 10: Example Output Screenshot 4

14. Testing Approach

Testing focuses on checking the main user operations, expected outputs, input validation, and persistence behavior. The provided output screenshots show successful complaint submission, complaint listing, and normal exit.

Test Case	Input / Action	Expected Result
TC-01	Submit a valid complaint	Complaint is added and a complaint ID is displayed.
TC-02	View all complaints	Stored complaint records are displayed.
TC-03	Search an existing complaint	Matching complaint record is displayed.
TC-04	Update complaint status	Selected complaint status is changed.
TC-05	Update complaint priority	Selected complaint priority is changed.
TC-06	Delete an existing complaint	Selected complaint is removed.
TC-07	View pending complaints	Submitted/In Progress complaints are displayed.
TC-08	View resolved complaints	Resolved complaints are displayed.
TC-09	Enter invalid menu/numeric input	Validation prevents invalid input from being accepted.
TC-10	Restart after saving records	Previously saved complaints can be loaded from the text file.
TC-11	Exit from main menu	Application terminates and displays the exit message.

15. Challenges Faced

- Designing a complaint structure that stores all required information clearly.
- Managing multiple complaint records dynamically using an ArrayList.
- Separating user-interface, validation, business logic, and file-storage responsibilities.
- Maintaining a consistent text-file format for saving and loading records.
- Handling invalid user input without terminating the application unexpectedly.
- Keeping search, update, and delete operations consistent with the stored complaint data.
- Designing documentation and diagrams that match the actual implementation.

16. Learnings & Key Takeaways

- Understood how classes and objects can represent real-world entities such as complaints.
- Applied encapsulation using private fields and accessor methods.
- Learned to use ArrayList for dynamic collections of objects.
- Applied file handling to create persistent records.
- Used exception handling and validation for safer user input.
- Learned how modular classes improve organization and maintainability.
- Understood the importance of system architecture, workflow, UML diagrams, and storage design in project documentation.
- Gained practical experience in structuring a Java project for GitHub-based submission.

17. Future Enhancements

- Add a graphical interface using Java Swing or JavaFX.
- Replace text-file storage with JDBC and a relational database such as MySQL.
- Add separate student and administrator/staff login roles.
- Allow staff to be assigned to specific complaints.
- Add email or notification support for status updates.
- Allow image attachments for maintenance issues.
- Create an administrator dashboard with charts and statistics.
- Add complaint timestamps and maintenance history.
- Deploy the system as a web or mobile application.

18. Conclusion

The Hostel Maintenance & Complaint Tracker provides a simple and organized approach to recording and managing hostel maintenance complaints. The system supports complaint submission, viewing, searching, updating, deleting, status-based filtering, and summary operations. Persistent storage through a text file allows complaint records to be retained between program executions.

From an academic perspective, the project demonstrates practical use of Java classes and objects, encapsulation, ArrayList collections, methods, file handling, input validation, and exception handling. The modular design also provides a clear foundation for future extensions such as a graphical interface, database integration, user authentication, and notifications.

19. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
- Java programming course notes and classroom material.
- Java Object-Oriented Programming concepts.
- Java Collections Framework concepts, especially ArrayList.
- Java file handling and exception handling concepts.
- Git and GitHub documentation and version-control concepts.